PES UNIVERSITY

100 feet Ring Road, BSK 3rd Stage, Bengaluru 560085

PES UNIVERSITY

Department of Computer Science and Engineering (AI & ML)

B. Tech. CSE — 5th Semester | Aug–Dec 2026

UE24CS351A
DATABASE MANAGEMENT SYSTEMS (DBMS)

PROJECT REPORT

PatentScan: Hybrid Patent Prior-Art Search & Claim Analysis System

Submitted by

SRN	Name	Section
PES1UG24AM067	Bhogala Srika	B
PES1UG24AM066	Bhavya k	B

GITHUB REPO: https://github.com/BHOGALA-SRIKA/PatentScan

Abstract

PatentScan is a web-based patent prior-art exploration and claim-analysis support system designed to help student innovators, patent researchers and R&D teams organize invention investigations and discover potentially relevant patent documents. The application combines structured filtering and keyword retrieval with embedding-based semantic retrieval and a hybrid search mode. PostgreSQL stores normalized application and patent records, while the pgvector extension stores 384-dimensional patent embeddings and supports vector similarity search. MongoDB is used for flexible document-oriented metadata and relationship-oriented records.

The system supports user registration and role-based access, investigation creation, claim-element capture, patent search, evidence organization and administrative reporting. The relational design separates users, patents, investigations, claim elements, selected investigation patents, evidence mappings, search logs and patent embeddings. Search and evidence results are intended to assist human review; they do not constitute a legal opinion or establish novelty, infringement or freedom to operate.

The implementation uses Next.js and React for the web interface, Next.js route handlers for backend APIs, PostgreSQL/pgvector for relational and vector data, MongoDB for document data, and a Python embedding service based on the Sentence Transformers model all-MiniLM-L6-v2. The report documents the system design, database schema, representative SQL/NoSQL operations, application architecture and evaluation status. Any feature marked as requiring verification must be supported with a live demonstration or execution evidence before final submission.

Table of Contents

Sl. No	Topic	Page No.
1.	Introduction a) Problem Statement	4
2.	User Requirements Specification- Tech Stack	5
3.	E-R Model a) List of Entities [labeling Weak ones] b) List non-simple Attributes c) ER Diagram Snapshot d) ERD from tool like draw.io	6
4.	Relational Model a) List of Relations that are not Entities in ER Model b) Schema Diagram [ensure light background image]	9
5.	SQL DDL Statements	10
6.	SQL DML Statements	11
7.	NoSQL/ Vector Database Commands	12
8.	Results a) Resulting Tables' Screenshots	14
9.	Next.js-Project Tree Structure	17
10.	Few sample Screenshots of application code and demo	18
11.	Evaluation Rubrics- Completion Status	22

1. Introduction

1.1 Background

Patent prior-art search is an important activity when assessing whether an invention may overlap with previously published technical disclosures. Patent documents contain structured fields such as publication number, dates, classifications and legal status, together with unstructured text such as titles, abstracts and claims. A useful search workflow therefore benefits from both structured database queries and text-based relevance retrieval.

1.2 Problem Statement

Researchers may need to search across a large and heterogeneous patent collection, compare documents using both exact terminology and conceptual similarity, and organize the findings against individual invention or claim elements. A workflow that relies only on exact keyword matching may miss documents that describe similar concepts using different terminology. A workflow based only on semantic similarity may make it difficult to apply precise filters such as publication date, classification or legal status. PatentScan addresses this workflow by integrating structured patent storage, keyword search, vector-based semantic search, hybrid retrieval and investigation-level evidence organization.

1.3 Objectives

Provide a web interface for user registration, login and patent investigation workflows.

Store patent bibliographic information and investigation data in a relational schema with keys and constraints.

Support keyword, semantic and hybrid patent retrieval modes.

Use vector embeddings to represent patent text and pgvector to support similarity search.

Allow users to create investigations and record invention descriptions and claim elements.

Organize candidate patents and evidence mappings for human review.

Provide administrative reports over available system data where the corresponding reporting queries are connected.

1.4 Scope and Limitations

PatentScan is an academic decision-support prototype. It is not a patentability determination engine, legal search opinion, infringement detector or substitute for professional patent counsel. Search ranking and claim-to-evidence suggestions are preliminary. A human reviewer must inspect the source patent, claims, prosecution history and relevant legal context. Coverage and freshness depend on the source data collected and imported into the database.

2. User Requirements Specification

2.1 User Roles

Role	Main capabilities	Access boundary
Researcher / User	Register and log in; create investigations; enter invention descriptions and claim elements; search patents; inspect results; organize evidence and view own investigation/search history.	Must not access other users' private investigations or user-level history.
Administrator	Access the administrative dashboard; review system-level patent and usage summaries; manage/inspect system-wide information as implemented.	Administrative access is restricted to the ADMIN role.

2.2 Functional Requirements

Authentication: validate credentials and issue a signed JWT stored in an authentication cookie.

Authorization: verify the authenticated user and role on protected API routes.

Patent search: accept a query and search mode; return ranked patent records with bibliographic fields.

Keyword retrieval: use PostgreSQL full-text search over title and abstract.

Semantic retrieval: obtain a 384-dimensional query embedding and compare it with stored patent vectors.

Hybrid retrieval: combine lexical and semantic signals to rank candidate patent records.

Investigation management: persist an investigation and its claim elements, and retrieve user-owned investigation records.

Evidence organization: associate claim elements with patent records and store an evidence level and reviewer note.

Reporting: summarize patent counts, classifications, legal status and recent records; user/admin reporting depends on the active report API and data source.

2.3 Non-functional Requirements

Data integrity through primary keys, foreign keys, unique constraints and CHECK constraints.

Secure password storage using password hashing rather than plaintext passwords.

Parameterized SQL statements for database operations.

Maintainable separation between frontend routes, API handlers and database connection modules.

Reasonable response time for bounded search result sets; search limits are capped by the API.

Traceable patent records through publication numbers and source URLs.

2.4 Technology Stack and Justification

Technology	Purpose	Justification
Next.js 16 / React 19 / TypeScript	Frontend pages and server/API routes	Provides a single application framework for UI and backend route handlers with typed application code.
Node.js / Next.js Route Handlers	Backend API layer	Handles authentication, request validation, search orchestration and database access.
PostgreSQL 17	Relational database	Suitable for structured patent, user, investigation and evidence data with SQL, constraints, joins and transactions.
pgvector	Vector extension for PostgreSQL	Stores VECTOR(384) embeddings and enables vector similarity operations alongside relational records.
MongoDB	Document database	Supports flexible document-oriented metadata and relationship records where document structure is more suitable than fixed relational columns.
Python + Sentence Transformers	Embedding service	Generates 384-dimensional text embeddings using all-MiniLM-L6-v2 for semantic retrieval.

NoSQL justification: MongoDB complements PostgreSQL rather than replacing it. Relational entities and integrity-sensitive transactions remain in PostgreSQL. Flexible investigation metadata and relationship-oriented documents can be stored as MongoDB documents. pgvector is an extension within PostgreSQL, not a separate NoSQL database.

3. E-R Model

3.1 Entity List

Entity	Primary key	Description / important attributes	Weak entity?
APP_USERS	user_id	full_name, email (unique), password_hash, role, created_at	No
PATENTS	patent_id	publication_number (unique), title, abstract, publication_date, priority_date, ipc_code, cpc_code, legal_status, source_url	No
INVESTIGATIONS	investigation_id	user_id (FK), title, invention_description, cutoff_date, status, created_at	No
CLAIM_ELEMENTS	element_id	investigation_id (FK), element_text, element_order, importance	No; independently keyed, dependent by FK
INVESTIGATION_PATENTS	(investigation_id, patent_id)	Composite key; relevance_score, selection_reason, added_at	Associative entity
EVIDENCE_MAPPINGS	mapping_id	element_id (FK), patent_id (FK), evidence_level, evidence_note, reviewed_by, created_at	No
SEARCH_LOGS	search_id	investigation_id (nullable FK), user_id (migration FK), query_text, search_type, results_count, duration_ms, domain, created_at	No
PATENT_EMBEDDINGS	patent_id (PK/FK)	embedding VECTOR(384), model_name, embedded_at	No; one-to-one extension of PATENTS

3.2 Relationships and Cardinalities

One APP_USERS record can own many INVESTIGATIONS; each investigation belongs to one user.

One INVESTIGATIONS record can contain many CLAIM_ELEMENTS; claim elements are deleted when their parent investigation is deleted (ON DELETE CASCADE).

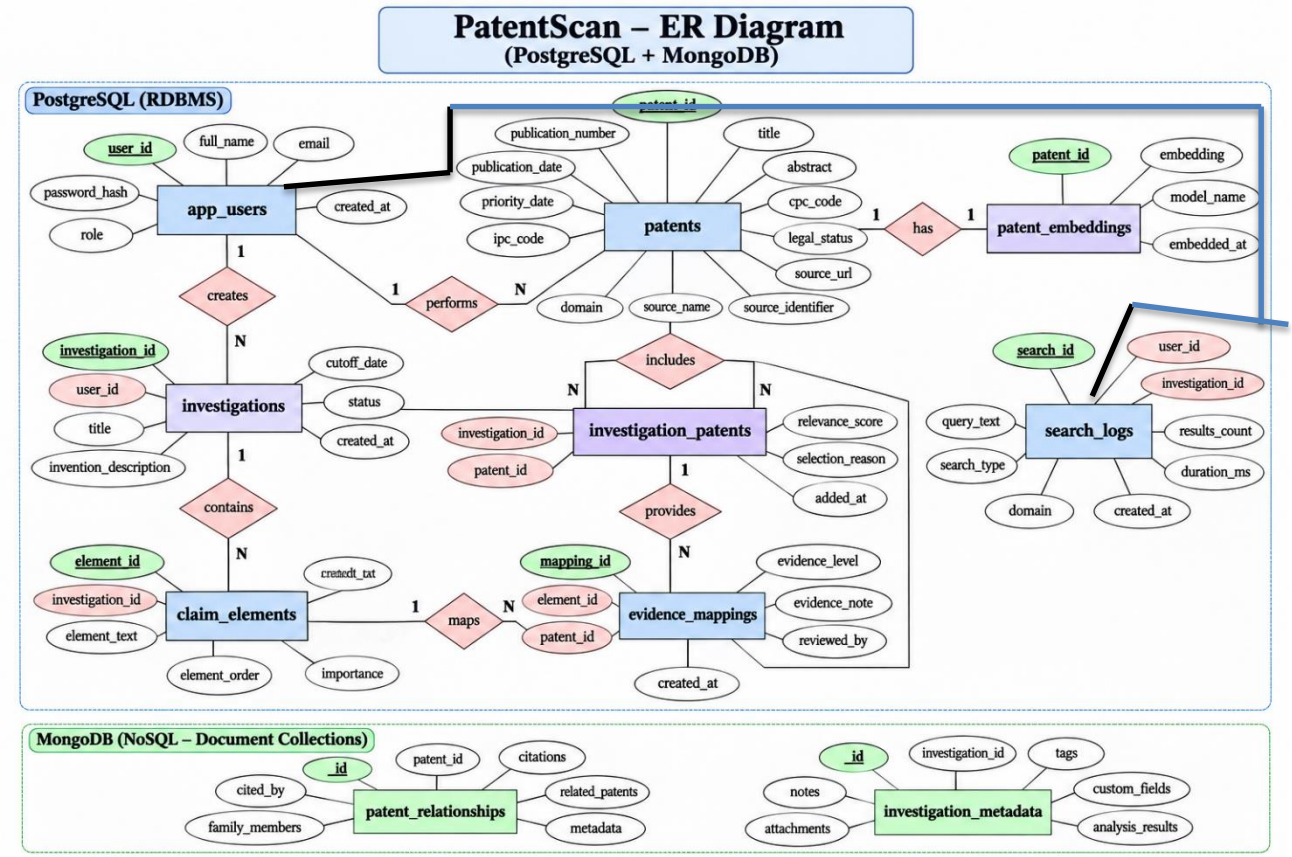
INVESTIGATIONS and PATENTS have a many-to-many relationship resolved through INVESTIGATION_PATENTS.

CLAIM_ELEMENTS and PATENTS are associated through EVIDENCE_MAPPINGS; the mapping records evidence level and notes.

One PATENTS record may have one PATENT_EMBEDDINGS record; patent_id is both primary key and foreign key.

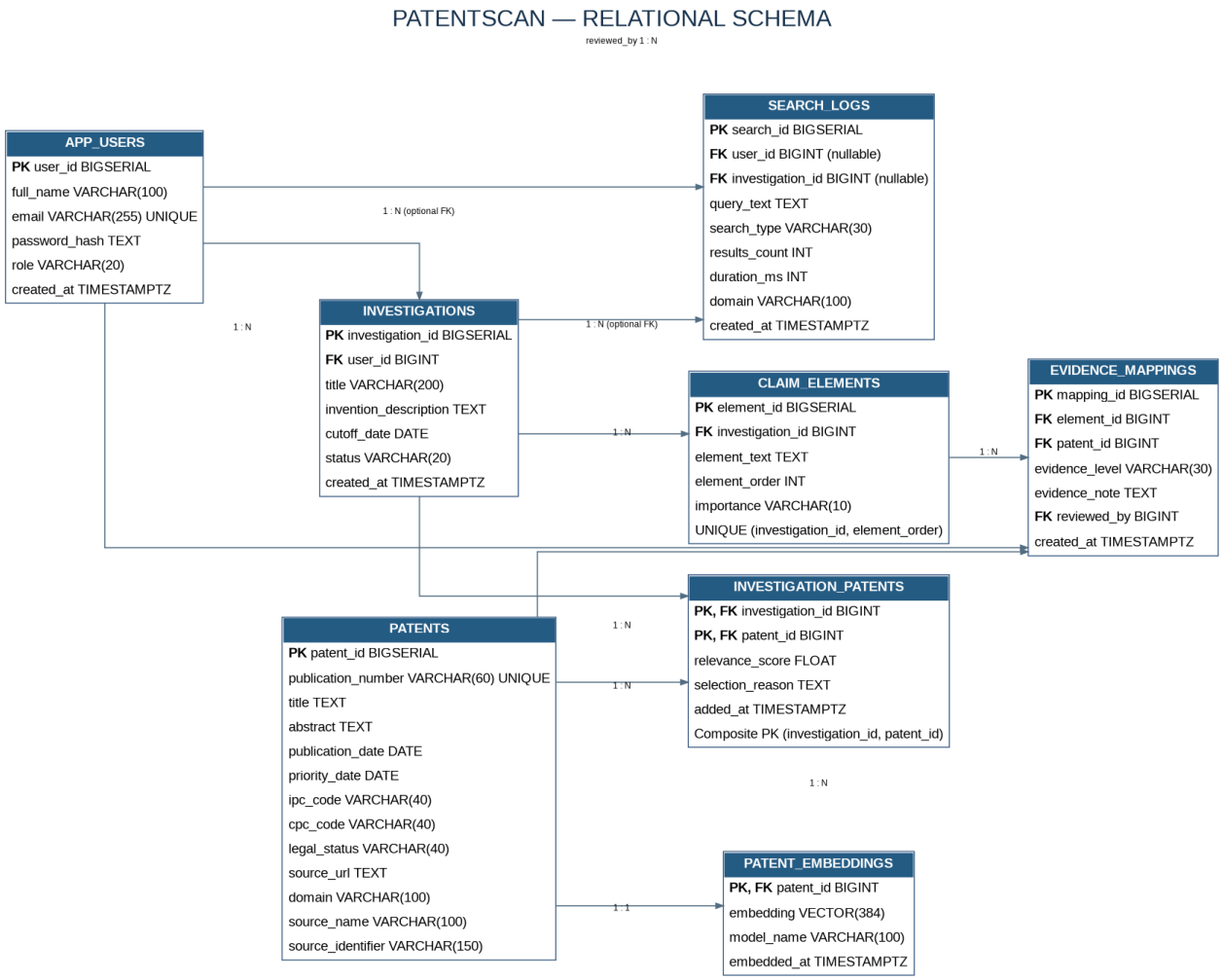
SEARCH_LOGS records a search event and may optionally be associated with an investigation; the additive migration also adds user_id and domain.

3.3 ER Diagram



4. Relational Model

4.1 Relational Schema



APP_USERS(user_id PK, full_name, email UNIQUE, password_hash, role, created_at)

PATENTS(patent_id PK, publication_number UNIQUE, title, abstract, publication_date, priority_date, ipc_code, cpc_code, legal_status, source_url, domain, source_name, source_identifier)

INVESTIGATIONS(investigation_id PK, user_id FK → APP_USERS.user_id, title, invention_description, cutoff_date, status, created_at)

CLAIM_ELEMENTS(element_id PK, investigation_id FK → INVESTIGATIONS.investigation_id, element_text, element_order, importance, UNIQUE(investigation_id, element_order))

INVESTIGATION_PATENTS(investigation_id PK/FK, patent_id PK/FK, relevance_score, selection_reason, added_at)

EVIDENCE_MAPPINGS(mapping_id PK, element_id FK → CLAIM_ELEMENTS.element_id, patent_id FK → PATENTS.patent_id, evidence_level, evidence_note, reviewed_by FK → APP_USERS.user_id, created_at)

SEARCH_LOGS(search_id PK, investigation_id nullable FK, user_id nullable FK, query_text, search_type, results_count, duration_ms, domain, created_at)

PATENT_EMBEDDINGS(patent_id PK/FK → PATENTS.patent_id, embedding VECTOR(384), model_name, embedded_at)

4.2 Relations Not Directly Represented as Independent Conceptual Entities

INVESTIGATION_PATENTS is an associative relation resolving the many-to-many relationship between investigations and patents. PATENT_EMBEDDINGS is a one-to-one technical extension of PATENTS for vector data. SEARCH_LOGS is an event/transaction-history relation. These relations are retained as database tables because they carry relationship attributes or operational data.

4.3 Keys, Constraints and Indexes

Primary keys are generated using BIGSERIAL for surrogate identifiers, except composite key INVESTIGATION_PATENTS(investigation_id, patent_id).

Unique constraints protect app_users.email, patents.publication_number and claim_elements(investigation_id, element_order).

CHECK constraints restrict role, investigation status, claim importance, search type, evidence level and non-negative counts/durations.

Foreign keys enforce ownership and referential integrity. ON DELETE CASCADE is used for dependent claim elements, investigation-patent links and evidence mappings.

Indexes support publication date, priority date, IPC/CPC classification, investigation ownership, claim-element lookup, search-log time/type/domain and patent full-text/vector retrieval.

4.4 Schema Diagram

Figure 1 on the preceding page is the relational schema/ER view. It reflects the supplied base DDL plus the additive migration that adds patent provenance/domain fields and user/domain columns to search_logs. Confirm the live database schema using information_schema before submission if additional migrations have been applied.

5. SQL DDL Statements

The following statements are representative excerpts from the supplied schema. They show database creation, constraints, relationships and vector support. The complete executable schema should be retained in the project repository as schema.sql or the migration files.

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE app_users (
  user_id BIGSERIAL PRIMARY KEY,
  full_name VARCHAR(100) NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  role VARCHAR(20) NOT NULL DEFAULT 'RESEARCHER'
    CHECK (role IN ('RESEARCHER','ADMIN')),
  created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE patents (
  patent_id BIGSERIAL PRIMARY KEY,
  publication_number VARCHAR(60) UNIQUE NOT NULL,
  title TEXT NOT NULL,
  abstract TEXT NOT NULL,
  publication_date DATE,
  priority_date DATE,
  ipc_code VARCHAR(40),
  cpc_code VARCHAR(40),
  legal_status VARCHAR(40),
  source_url TEXT
);

CREATE TABLE investigations (
  investigation_id BIGSERIAL PRIMARY KEY,
  user_id BIGINT NOT NULL REFERENCES app_users(user_id),
  title VARCHAR(200) NOT NULL,
  invention_description TEXT NOT NULL,
  cutoff_date DATE,
  status VARCHAR(20) NOT NULL DEFAULT 'OPEN'
    CHECK (status IN ('OPEN','COMPLETED')),
  created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE claim_elements (
  element_id BIGSERIAL PRIMARY KEY,
  investigation_id BIGINT NOT NULL
    REFERENCES investigations(investigation_id) ON DELETE CASCADE,
  element_text TEXT NOT NULL,
  element_order INT NOT NULL CHECK (element_order > 0),
  importance VARCHAR(10) NOT NULL DEFAULT 'MEDIUM'
    CHECK (importance IN ('HIGH','MEDIUM','LOW')),
  UNIQUE (investigation_id, element_order)
);

CREATE TABLE patent_embeddings (
  patent_id BIGINT PRIMARY KEY REFERENCES patents(patent_id)
    ON DELETE CASCADE,
  embedding VECTOR(384) NOT NULL,
  model_name VARCHAR(100) NOT NULL,
  embedded_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX patents_text_search_idx ON patents USING GIN
  (to_tsvector('english', coalesce(title,'') || ' ' || coalesce(abstract,'')));

CREATE INDEX patent_embeddings_hnsw_idx ON patent_embeddings
  USING hnsw (embedding vector_cosine_ops);
```

Additional DDL in the complete schema defines investigation_patents, evidence_mappings and search_logs, along with their foreign keys, CHECK constraints and indexes. The additive migration adds domain/source provenance fields to patents, and user_id/domain to search_logs, and relaxes investigation_id to nullable for searches not attached to an investigation.

```sql
-- Additive and data-preserving. Safe to run more than once.
-- Run against the existing `patentscan` database after taking a backup.

BEGIN;

ALTER TABLE patents
    ADD COLUMN IF NOT EXISTS domain VARCHAR(80) NOT NULL DEFAULT 'Needs Review',
    ADD COLUMN IF NOT EXISTS source_name VARCHAR(120),
    ADD COLUMN IF NOT EXISTS source_identifier VARCHAR(160),
    ADD COLUMN IF NOT EXISTS source_license VARCHAR(160),
    ADD COLUMN IF NOT EXISTS source_metadata JSONB NOT NULL DEFAULT '{}'::jsonb,
    ADD COLUMN IF NOT EXISTS embedding_status VARCHAR(16) NOT NULL DEFAULT 'PENDING',
    ADD COLUMN IF NOT EXISTS embedding_text_hash CHAR(64),
    ADD COLUMN IF NOT EXISTS embedding_error TEXT;

-- Backfill provenance for existing records without inventing a source.
UPDATE patents
SET source_identifier = publication_number
WHERE source_identifier IS NULL;

-- Existing vectors are retained and marked complete; records without vectors stay pending.
UPDATE patents p
SET embedding_status = 'COMPLETED'
WHERE EXISTS (SELECT 1 FROM patent_embeddings pe WHERE pe.patent_id = p.patent_id);

-- Keep the current source_url untouched. A URL is not proof of a source API.
CREATE INDEX IF NOT EXISTS patents_domain_idx ON patents(domain);
CREATE INDEX IF NOT EXISTS patents_source_name_idx ON patents(source_name);
CREATE UNIQUE INDEX IF NOT EXISTS patents_source_identifier_uq
    ON patents(source_name, source_identifier)
    WHERE source_name IS NOT NULL AND source_identifier IS NOT NULL;
```

6. SQL DML Statements and Queries

6.1 Insert and Read Operations

```sql
-- Insert a researcher account (password_hash must be a bcrypt hash)
INSERT INTO app_users(full_name, email, password_hash, role)
VALUES ('Example Researcher', 'researcher@example.edu', '<bcrypt_hash>', 'RESEARCHER');


-- Insert a patent record
INSERT INTO patents(publication_number, title, abstract, publication_date,
                    priority_date, ipc_code, cpc_code, legal_status, source_url)
VALUES ('EXAMPLE-2026-001', 'Example title', 'Example abstract',
        DATE '2026-01-15', DATE '2024-06-01', 'G06F', 'G06F16/00',
        'Published', 'https://patents.google.com/');


-- Retrieve recent patents
SELECT publication_number, title, publication_date, ipc_code, legal_status
FROM patents
ORDER BY publication_date DESC NULLS LAST
LIMIT 10;
```

6.2 Join Query

```sql
SELECT i.investigation_id, i.title, u.full_name,
       COUNT(DISTINCT ce.element_id) AS claim_element_count
FROM investigations i
JOIN app_users u ON u.user_id = i.user_id
LEFT JOIN claim_elements ce ON ce.investigation_id = i.investigation_id
GROUP BY i.investigation_id, i.title, u.full_name
ORDER BY i.created_at DESC;
```

6.3 Nested / Correlated Query

```
-- Patents with at least one evidence mapping
SELECT p.publication_number, p.title
FROM patents p
WHERE EXISTS (
  SELECT 1
  FROM evidence_mappings em
  WHERE em.patent_id = p.patent_id
);
```

6.4 CTE / Recursive Query Example

```
-- CTE: summarize patents selected per investigation
WITH patent_totals AS (
  SELECT investigation_id, COUNT(*) AS selected_patents
  FROM investigation_patents
  GROUP BY investigation_id
)
SELECT i.investigation_id, i.title,
       COALESCE(pt.selected_patents, 0) AS selected_patents
FROM investigations i
LEFT JOIN patent_totals pt
  ON pt.investigation_id = i.investigation_id;


-- Recursive CTE example: hierarchical category traversal
-- Run only if a category(parent_id, category_id, name) table exists.
WITH RECURSIVE category_tree AS (
  SELECT category_id, parent_id, name, 0 AS depth
  FROM category WHERE parent_id IS NULL
  UNION ALL
  SELECT c.category_id, c.parent_id, c.name, t.depth + 1
  FROM category c JOIN category_tree t
    ON c.parent_id = t.category_id
)
SELECT * FROM category_tree ORDER BY depth, name;
```

The recursive CTE is a demonstration pattern for hierarchical data and is not claimed as an existing PatentScan table or deployed query. Remove it if the course evaluator expects only executed project SQL.

6.5 Update, Delete and Transaction

```
UPDATE investigations
SET status = 'COMPLETED'
WHERE investigation_id = 1;


DELETE FROM investigation_patents
WHERE investigation_id = 1 AND patent_id = 10;


BEGIN;
INSERT INTO investigations(user_id, title, invention_description)
VALUES (1, 'Prototype investigation', 'Description...')
RETURNING investigation_id;
-- Insert related claim_elements using the returned investigation_id.
COMMIT;
```

The claim-analysis API uses a database transaction when it inserts an investigation and its claim elements, committing on success and rolling back on failure. Replace example identifiers with valid IDs from the live database before running the sample statements.

7. NoSQL / Vector Database Commands

7.1 MongoDB

MongoDB is used for flexible document-oriented records and relationship-oriented metadata. The supplied application route confirms a collection named investigation_metadata, where an authenticated user's investigation metadata is inserted and retrieved using user_id.

```
// Inspect the active database and collections
show dbs
use patentscan
show collections


// Example shape based on the application route's fields
db.investigation_metadata.find(
  { user_id: 1 },
  { investigation_name: 1, technology_area: 1,
    search_method: 1, status: 1, created_at: 1 }
).sort({ created_at: -1 });


// Count saved metadata records for one user
db.investigation_metadata.countDocuments({ user_id: 1 });
```

The actual collection names and fields used for patent citation/family relationship documents must be confirmed from the live MongoDB database before adding further commands. Do not present guessed collection names as executed commands.

7.2 pgvector

The patent_embeddings table stores one 384-dimensional vector per patent. The project's embedding service uses Sentence Transformers all-MiniLM-L6-v2. A query is embedded by the Python service and the resulting vector is passed to PostgreSQL for similarity ranking.

```
CREATE EXTENSION IF NOT EXISTS vector;


-- Example cosine-distance query; $1 is a 384-dimensional vector literal.
SELECT p.patent_id, p.publication_number, p.title,
       pe.embedding <=> $1::vector AS cosine_distance
FROM patent_embeddings pe
JOIN patents p ON p.patent_id = pe.patent_id
ORDER BY pe.embedding <=> $1::vector
LIMIT 10;


-- Verify vector index
SELECT indexname, indexdef
FROM pg_indexes
WHERE tablename = 'patent_embeddings';
```

The API exposes keyword, semantic and hybrid search modes. Keyword mode uses PostgreSQL full-text search over title and abstract. Semantic and hybrid modes call the embedding service and use the stored vectors. Exact score-fusion weights and ranking behavior should be described from the current route implementation and verified with a live query.

8. Application Architecture and Results Tables Screenshot

```
patentscan=# \dt
                List of relations
 Schema |          Name          | Type  |  Owner
--------+------------------------+-------+----------
 public | app_users              | table | postgres
 public | claim_elements         | table | postgres
 public | evidence_mappings      | table | postgres
 public | investigation_patents  | table | postgres
 public | investigations         | table | postgres
 public | patent_embeddings      | table | postgres
 public | patents                | table | postgres
 public | search_logs            | table | postgres
(8 rows)
```

```
                                    Table "public.patents"
      Column       |          Type          | Collation | Nullable |                 Default
-------------------+------------------------+-----------+----------+-------------------------------
-------------
 patent_id         | bigint                 |           | not null | nextval('patents_patent_id_se
q'::regclass)
 publication_number | character varying(60) |           | not null |
 title             | text                   |           | not null |
 abstract          | text                   |           | not null |
 publication_date  | date                   |           |          |
 priority_date     | date                   |           |          |
 ipc_code          | text                   |           |          |
 cpc_code          | text                   |           |          |
 legal_status      | text                   |           |          |
 source_url        | text                   |           |          |
 domain            | character varying(100) |           |          |
 source_name       | character varying(120) |           |          |
 source_identifier | character varying(160) |           |          |
 source_license    | character varying(160) |           |          |
 source_metadata   | jsonb                  |           | not null | '{}'::jsonb
 embedding_status  | character varying(16)  |           | not null | 'PENDING'::character varying
 embedding_text_hash | character(64)        |           |          |
 embedding_error   | text                   |           |          |
Indexes:
```

```
                                        Table "public.investigations"
       Column         |           Type           | Collation | Nullable |                        D
efault
----------------------+--------------------------+-----------+----------+--------------------------
------------------------------
 investigation_id      | bigint                   |           | not null | nextval('investigations_i
nvestigation_id_seq'::regclass)
 user_id               | bigint                   |           | not null |
 title                 | character varying(200)   |           | not null |
 invention_description | text                     |           | not null |
 cutoff_date           | date                     |           |          |
 status                | character varying(20)    |           | not null | 'OPEN'::character varying
 created_at            | timestamp with time zone |           | not null | CURRENT_TIMESTAMP
Indexes:
    "investigations_pkey" PRIMARY KEY, btree (investigation_id)
    "idx_investigations_user" btree (user_id)
    "investigations_user_idx" btree (user_id)
Check constraints:
    "investigations_status_check" CHECK (status::text = ANY (ARRAY['OPEN'::character varying, 'COMPL
ETED'::character varying]::text[]))
Foreign-key constraints:
    "investigations_user_id_fkey" FOREIGN KEY (user_id) REFERENCES app_users(user_id)
Referenced by:
    TABLE "claim_elements" CONSTRAINT "claim_elements_investigation_id_fkey" FOREIGN KEY (investigat
ion_id) REFERENCES investigations(investigation_id) ON DELETE CASCADE
```

```
                                      Table "public.evidence_mappings"
     Column     |           Type           | Collation | Nullable |                       Default

----------------+--------------------------+-----------+----------+---------------------------------
---------------------
 mapping_id     | bigint                   |           | not null | nextval('evidence_mappings_mappi
ng_id_seq'::regclass)
 element_id     | bigint                   |           | not null |
 patent_id      | bigint                   |           | not null |
 evidence_level | character varying(10)    |           | not null |
 evidence_note  | text                     |           |          |
 reviewed_by    | bigint                   |           |          |
 created_at     | timestamp with time zone |           | not null | CURRENT_TIMESTAMP
Indexes:
    "evidence_mappings_pkey" PRIMARY KEY, btree (mapping_id)
    "evidence_mappings_element_id_patent_id_key" UNIQUE CONSTRAINT, btree (element_id, patent_id)
    "evidence_mappings_element_idx" btree (element_id)
    "idx_evidence_level" btree (evidence_level)
Check constraints:
    "evidence_mappings_evidence_level_check" CHECK (evidence_level::text = ANY (ARRAY['EXPLICIT'::ch
aracter varying, 'PARTIAL'::character varying, 'NONE'::character varying]::text[]))
Foreign-key constraints:
    "evidence_mappings_element_id_fkey" FOREIGN KEY (element_id) REFERENCES claim_elements(element_i
d) ON DELETE CASCADE
    "evidence_mappings_patent_id_fkey" FOREIGN KEY (patent_id) REFERENCES patents(patent_id) ON DELE
```

```
                                    Table "public.search_logs"
      Column      |           Type           | Collation | Nullable |                  Default
------------------+--------------------------+-----------+----------+-----------------------------
----------------
 search_id        | bigint                   |           | not null | nextval('search_logs_search_id
_seq'::regclass)
 investigation_id | bigint                   |           |          |
 query_text       | text                     |           | not null |
 search_type      | character varying(10)    |           | not null |
 results_count    | integer                  |           | not null | 0
 duration_ms      | integer                  |           |          |
 created_at       | timestamp with time zone |           | not null | CURRENT_TIMESTAMP
 user_id          | bigint                   |           |          |
 domain           | character varying(100)   |           |          |
Indexes:
    "search_logs_pkey" PRIMARY KEY, btree (search_id)
    "idx_search_logs_created" btree (created_at)
    "search_logs_created_idx" btree (created_at)
    "search_logs_domain_idx" btree (domain)
    "search_logs_type_idx" btree (search_type)
    "search_logs_user_created_idx" btree (user_id, created_at DESC)
    "search_logs_user_idx" btree (user_id)
Check constraints:
    "search_logs_duration_ms_check" CHECK (duration_ms >= 0)
:
```

8.1 High-level Architecture

Layer	Components	Responsibilities
Presentation	Next.js / React pages	Login, registration, researcher dashboard, admin dashboard, search and investigation UI.
Application/API	Next.js route handlers; JWT auth middleware	Request validation, authentication, authorization, search orchestration and report endpoints.
Relational data	PostgreSQL	Users, patents, investigations, claim elements, evidence mappings, search logs and embedding metadata.
Vector retrieval	pgvector + embedding service	Store 384-dimensional vectors and rank candidate patents by vector distance.
Document data	MongoDB	Flexible investigation metadata and relationship-oriented documents.
Data ingestion	Python collector/import scripts	Collect/import patent records and generate embeddings; ingestion should respect source policies and rate limits.

8.2 Main Workflow

User registers or logs in; backend validates credentials and issues a signed JWT.

User creates an investigation and provides an invention description and claim elements.

User submits a patent search query and selects keyword, semantic or hybrid mode.

Keyword mode performs PostgreSQL full-text search; semantic/hybrid modes obtain an embedding and use pgvector retrieval.

Candidate patent records are reviewed and may be associated with an investigation.

Claim elements and evidence mappings organize preliminary evidence for later human review.

Administrative reports summarize available system data.

8.3 Results and Verification

The source code supplied for this report confirms the presence of API routes for login, logout, admin login, patent search, claim analysis, evidence mappings, investigations and patent reports. The PostgreSQL schema defines

eight principal relational tables and the vector embedding table. The report API in the supplied source returns patent totals, classification counts, recent patents and legal-status counts.

The supplied project history also records a previous database snapshot of 163 patent records and 163 embeddings, and a MongoDB citation-relationship count of 5,424. These are historical project figures, not a live query result captured for this report. Re-run COUNT(*) and the corresponding MongoDB count command before submission if you intend to quote current totals.

9. Next.js Project Tree Structure and Demonstration

9.1 Project Tree (selected files)

```
patentscan-app/
├── app/
│   ├── admin/
│   │   ├── dashboard/
│   │   └── login/
│   ├── api/
│   │   ├── admin/login/
│   │   ├── login/
│   │   ├── logout/
│   │   ├── patents/search/
│   │   ├── patents/reports/
│   │   ├── claim-analysis/
│   │   ├── evidence-mappings/
│   │   └── investigations/
│   ├── dashboard/
│   ├── login/
│   ├── register/
│   └── researcher/dashboard/
├── lib/
│   ├── auth.ts
│   ├── authMiddleware.ts
│   ├── postgres.ts
│   └── mongodb.ts
├── db/
│   └── migrations/
├── scripts/
│   └── patent_collector.py
├── data/
├── package.json
└── .env.example
```

This is a selected tree, not a complete filesystem listing. The actual repository may contain additional pages, components, configuration files and scripts.

10.1 Running the Application

```
# Install dependencies
npm install


# Configure environment variables in .env.local
# DATABASE_URL, MONGODB_URI, MONGODB_DATABASE, JWT_SECRET


# Start PostgreSQL/pgvector and MongoDB services as configured
# Start the embedding service on the endpoint expected by the API (127.0.0.1:8001)


# Run the Next.js development server
npm run dev


# Open in browser
http://localhost:3000
```

10. 2 Application Screenshots

Figure 2. Landing page / project home

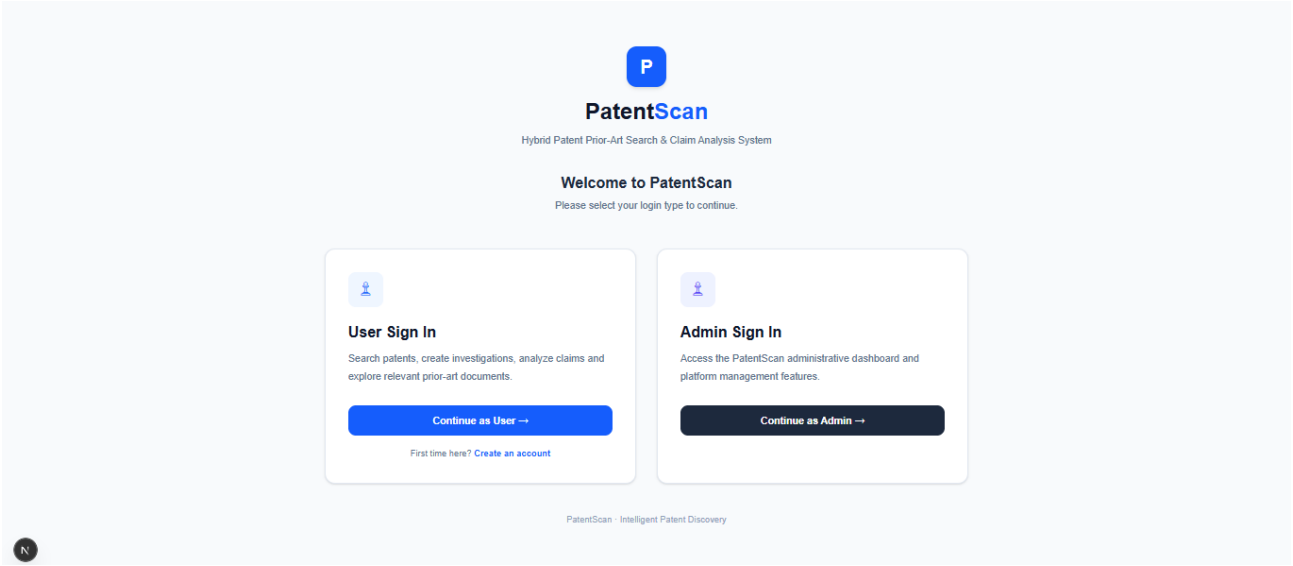


Figure 3. User registration and login

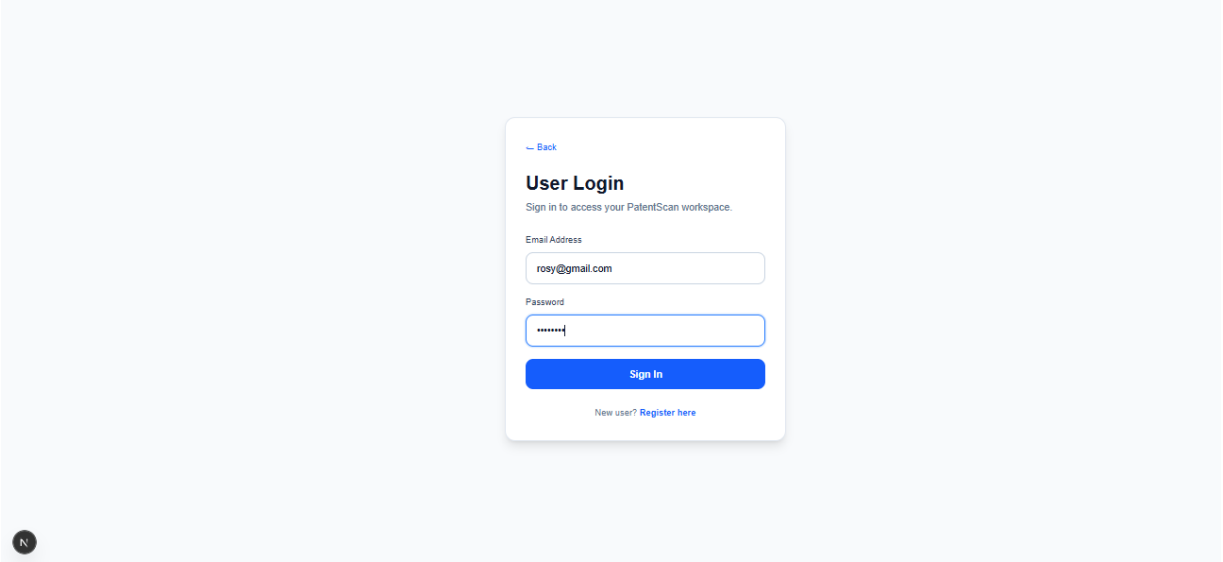


Figure 4. Researcher dashboard and patent search

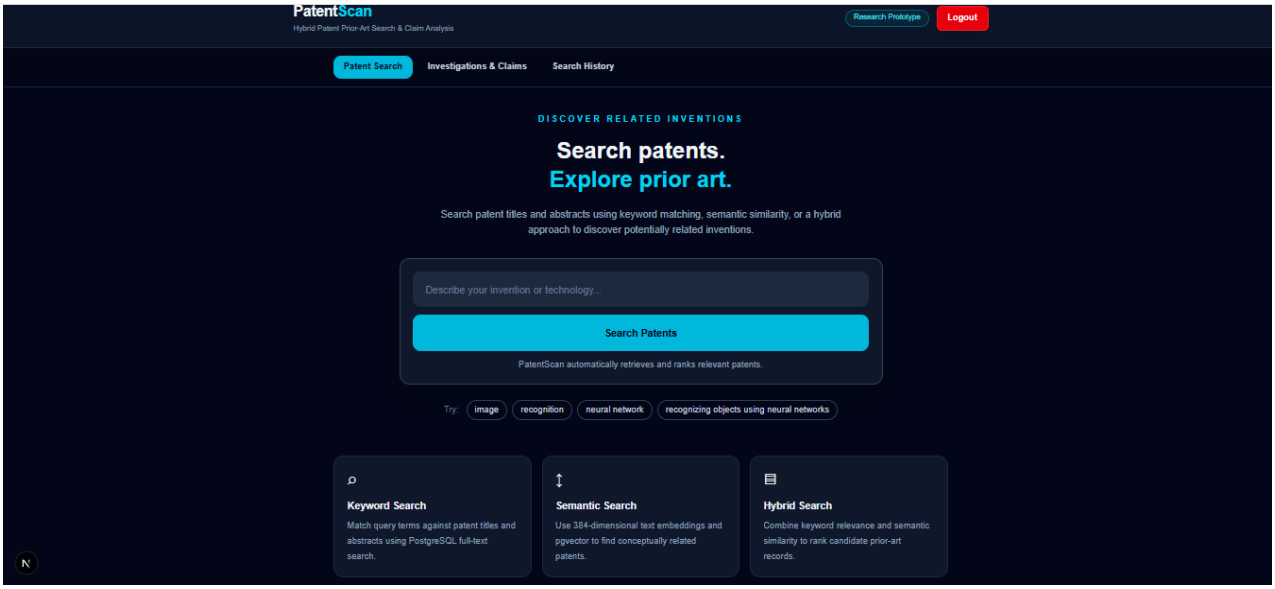


Figure 5. Search results showing hybrid modes

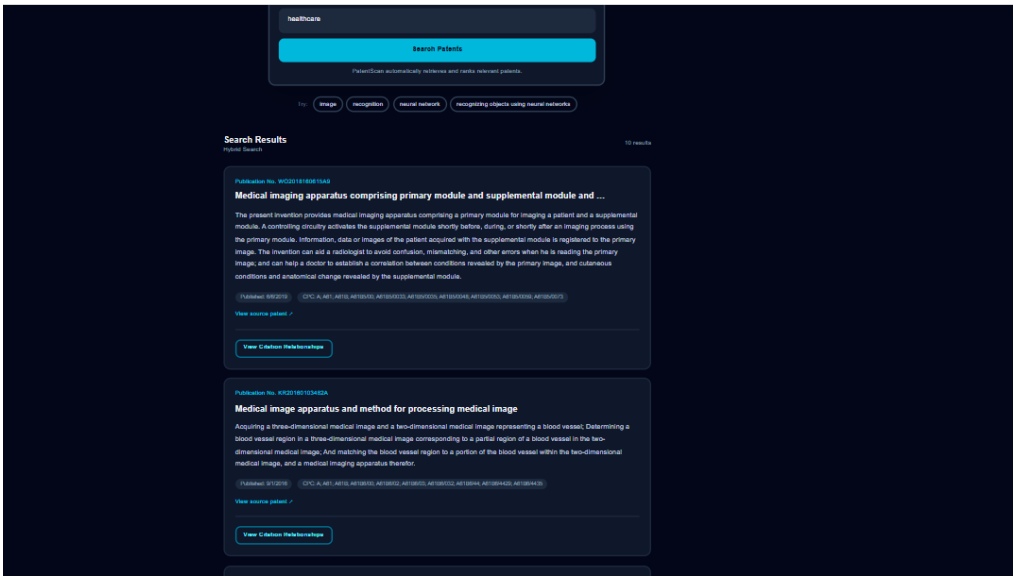


Figure 6. Investigation and claim-element workflow

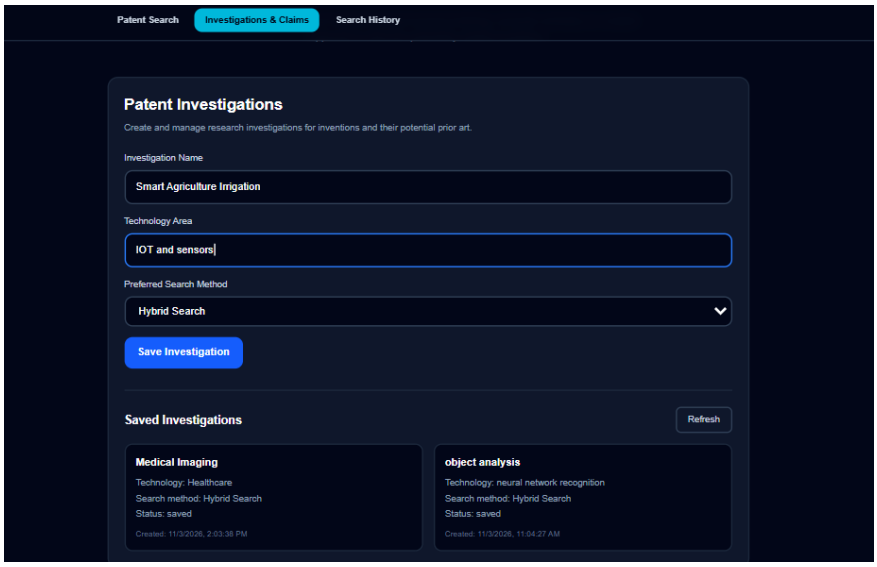


Figure 7. Evidence mapping view

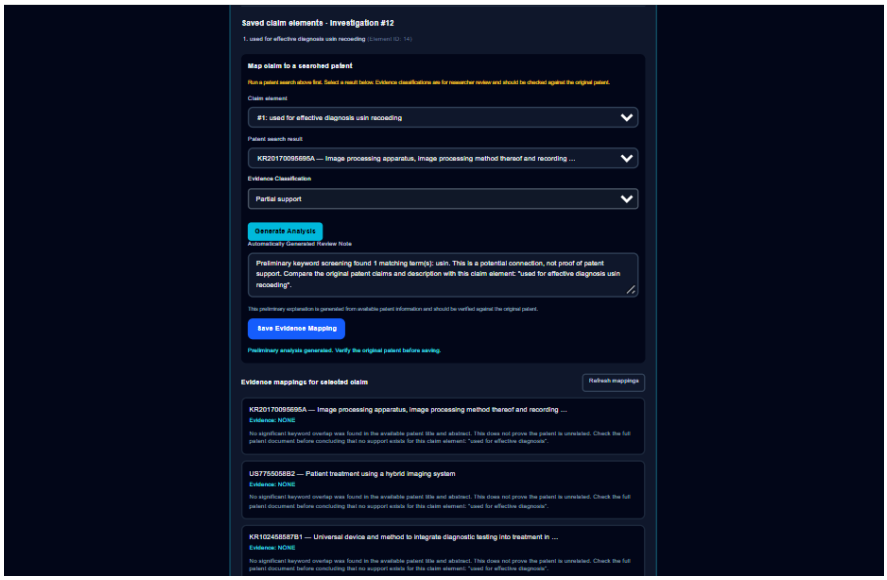


Figure 8. User Search History

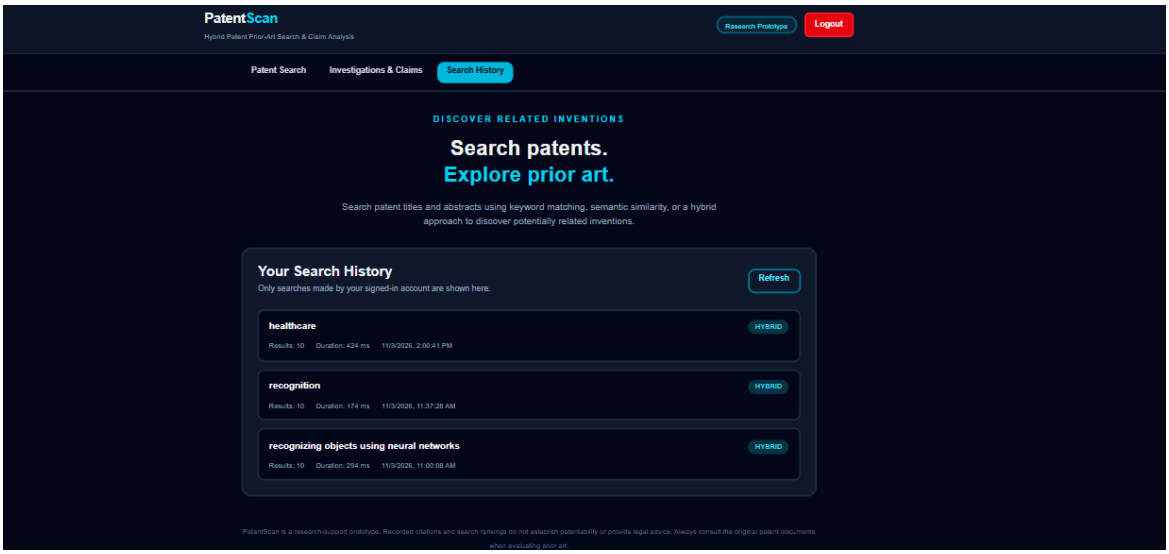
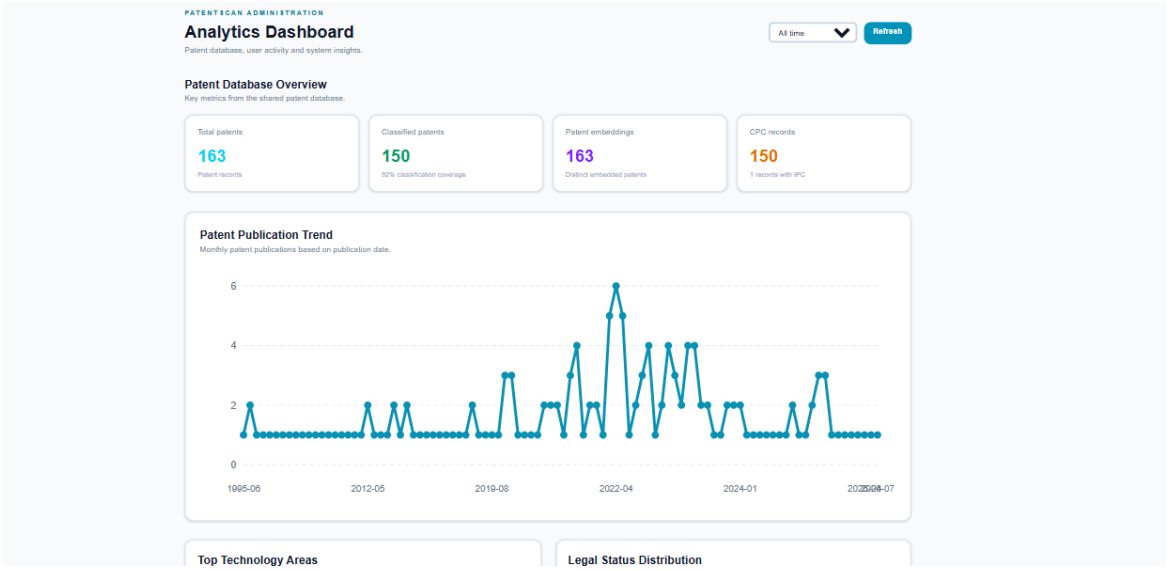


Figure 9. Admin dashboard and reports



11.1 Demonstration Sequence

Show the landing page and explain the prior-art search problem.
Register/login as a researcher and show role-restricted access.
Create an investigation and add claim elements.
Run one keyword query, then semantic and hybrid queries if the embedding service is running.
Open a candidate patent and explain the bibliographic fields and source link.
Show evidence mapping as preliminary organization, not a legal conclusion.
Log in as administrator and show the patent report dashboard.
Show the relational schema, vector table/index and MongoDB document collection.

11. Evaluation Rubrics and Completion Status

The Level 3 evaluation sheet distributes 50 marks across seven criteria. The table below reproduces the rubric categories and provides an evidence-oriented self-check. It does not claim marks awarded by the faculty.

#	Criteria	Evidence to present	Status
1	Database Design	ER diagram; relational schema; keys, relationships and constraints.	Schema documented; verify ERD against live schema.
2	Database Creation	PostgreSQL DDL, NoSQL/Vector database creation and technology justification.	DDL and technology rationale documented; attach execution screenshots.
3	SQL Operations and Queries	DML, nested/correlated/recursive queries and joins.	Representative queries included; attach actual execution outputs; recursive example is illustrative.
4	Advanced Database Concepts	Procedure/function, trigger, CTE and NoSQL/Vector RUD features.	CTE example included; procedure/trigger implementation evidence not verified in supplied source.
5	Web Application and Database Integration	Next.js interface, backend/API and database integration.	Routes and DB modules present in supplied source; attach current app/API screenshots.
6	Authentication and Authorization	Frontend/backend authentication, at least two roles and database role-based access.	JWT and role checks present in source; verify live role restriction and DB constraints.
7	Documentation, Git/GitHub, Demonstration, Testing and Originality	Documentation, GitHub, individual contribution, modification/viva and test evidence.	Report draft prepared; add repository URL, member contributions and test evidence.

References / Technical Resources

PostgreSQL Documentation — https://www.postgresql.org/docs/

pgvector — https://github.com/pgvector/pgvector

MongoDB Manual — https://www.mongodb.com/docs/manual/

Next.js Documentation — https://nextjs.org/docs

Sentence Transformers Documentation — https://www.sbert.net/

Google Patents — https://patents.google.com/ (patent discovery/source links; observe applicable terms and access policies)